

Testy ręczne plugin-2 – środowisko lokalne w stylu VPS

Cel: sprawdzić na żywo, że wtyczka renderuje podstronę „Nasze motory”, a scraper (automatyzacja) realnie pobiera oferty z poleasingowe.pl i zapisuje je do osobnej bazy.

Zasada: dwie niezależne warstwy (lepsze niż testowanie wszystkiego naraz)

Rozdzielamy test na dwie warstwy, żeby od razu wiedzieć co nie działa:

Warstwa	Co testuje	Jak
A. Automatyzacja (scraper -> MySQL)	działy 1A-7: crawl, parsowanie, normalizacja, audyt, dedup, zapis, anty-SSRF	uruchomienie scrapera na hoście
B. Front (MySQL -> WordPress)	wtyczka PHP: podstrona, filtry, SEO, cache, motyw	przeglądarka + narzędzia SEO

Topologia jest identyczna jak na produkcji: scraper (proces na hoście) pisze do MySQL kontem polea; WordPress czyta tę samą bazę kontem polea_ro (tylko SELECT).

0. Wymagania

- Docker + Docker Compose (WordPress i MySQL bez instalacji lokalnie).
- Python 3.9+ na hoście (do scrapera). Sprawdź: `python3 --version`.
- Dostęp do internetu (scraper pobiera z poleasingowe.pl).

1. Środowisko od zera (Docker) – warstwa B infrastruktura

Z katalogu repo:

```
docker compose -f deploy/local-test/docker-compose.yml up -d
```

To startuje:

- MySQL 8 na 127.0.0.1:3306 – od razu z bazą polea oraz kontami polea (zapis) i polea_ro (read-only) – patrz `deploy/local-test/initdb/00-init.sql`.
- WordPress na `<http://localhost:8080>` – ze wstrzykniętymi stałymi `POLEA_DB_*` (nie musisz ręcznie edytować `wp-config.php`) i włączonym `WP_DEBUG_LOG`.

Poczekaj ~20 s na wstanie bazy (`docker compose ... ps -> healthy`), potem:

1. Wejść na `<http://localhost:8080>` -> dokończ instalator WordPressa (tytuł, login, hasło).
2. Ustaw przyjazne odnośniki (wymagane dla ładnych URL-i pojedynczego motocykla i sitemapy):
Ustawienia -> Bezpośrednie odnośniki -> Nazwa wpisu -> Zapisz.

Załaduj schemat tabel w bazie polea

```
docker compose -f deploy/local-test/docker-compose.yml \
  exec -T db mysql -uroot -proot_dev polea < db/schema.sql
```

Weryfikacja:

```
docker compose -f deploy/local-test/docker-compose.yml \
  exec db mysql -uroot -proot_dev -e "SHOW TABLES;" polea
# oczekiwane: polea_motocykle, polea_zdjecia
```

2. Warstwa A – scraper LIVE (automatyzacja)

Scraper uruchamiasz na hoście (nie w kontenerze), pisząc do opublikowanego portu MySQL.

2.1. Przygotuj środowisko Pythona

```
cd "<repo>"
python3 -m venv .venv-test
. .venv-test/bin/activate
pip install requests PyMySQL
```

2.2. Poświadczenia bazy (konto zapisu polea)

```
export POLEA_DB_HOST=127.0.0.1
export POLEA_DB_PORT=3306
export POLEA_DB_USER=polea
export POLEA_DB_PASSWORD=polea_dev_pass
export POLEA_DB_NAME=polea
```

2.3. TEST PARSERA bez zapisu (kluczowy pierwszy krok)

Najpierw sprawdź, czy parser radzi sobie z realnym HTML-em źródła – bez dotykania bazy:

```
python3 -m scraper.main --limit 3 --dry-run -v
```

Co obserwować w logu:

- Strona 1: +N lotów -> dział 1A (crawl) działa,
- [i/N] OK <marka> <model> -> dział 1B/2/5 (szczegóły, normalizacja, audyt) OK,
- brak Audyt odrzucił masowo -> reguły twarde przechodzą,
- jeśli zobaczysz Odrzucono X/Y (>60%) – możliwa zmiana formatu źródła -> parser wymaga aktualizacji regexów (źródło zmieniło HTML). To celowy bezpiecznik – nie błąd wtyczki.

2.4. Zapis do bazy (mały wolumen)

```
python3 -m scraper.main --limit 20 -v
# log końcowy: "Zapis: N lotów; reconcile: M zamkniętych."
```

Weryfikacja w bazie:

```
docker compose -f deploy/local-test/docker-compose.yml \
  exec db mysql -uroot -proot_dev -e \
  "SELECT COUNT(*) motocykli FROM polea_motocykle; SELECT COUNT(*) zdjec FROM polea_zdjecia;" polea
```

Zaczynaj od --limit – nie obciążaj źródła. Pełny import (python3 -m scraper.main) uruchom dopiero, gdy warstwa B działa.

3. Warstwa A – automatyzacja „styl VPS” (cron lokalnie)

Chcemy zobaczyć cykliczny import bez prawdziwego serwera. Dwie opcje:

Opcja 1: crontab użytkownika (najprostsza)

Utwórz plik env ~/polea.env (wartości jak w 2.2) i wpis crona (crontab -e):

```
* /30 * * * * set -a; . $HOME/polea.env; set +a; cd <repo> && <repo>/venv-test/bin/python3 -m scraper.main >> $HOME/polea-import.log 2>&1
```

- Podgląd: tail -f ~/polea-import.log.
- Test blokady pojedynczej instancji (flock): uruchom scraper dwa razy naraz – drugi ma napisać Inny import już działa (lock ...) – przerywam. i zakończyć się kodem 1.

Opcja 2: systemd (bliżej produkcji)

W repo są gotowe jednostki: deploy/systemd/polea-import.service + .timer (oraz wariant cron deploy/cron/polea-import.cron). Na VPS instaluje się je wg deploy/README.md. Lokalnie możesz je odpalić jako systemd --user po dostosowaniu ścieżek.

Docelowo na VPS: import co 4 h (timer/cron już to ustawia), poświadczenia w /etc/polea.env (chmod 600), konto polea_ro dla WordPressa, backup wg deploy/backup/polea-backup.sh. Szczegóły: deploy/README.md.

4. Warstwa B – instalacja wtyczki i PEŁNA CHECKLISTA frontu

4.1. Aktywacja

Wtyczki -> Importer Motocykli (poleasingowe.pl) -> Włącz. Wtyczka jest już podmontowana z repo.

Przy aktywacji dzieje się automatycznie (patrz includes/activation.php):

- tworzy się strona „Nasze motory” (/nasze-motory/) ze shortcode [motocykle],
- pozycja dopina się do menu (motyw klasyczny lub blokowy),
- rejestrują się ładne URL-e i sitemap.

4.2. Checklista (odhacz każdy punkt)

#	Obszar	Krok	Oczekiwany wynik
1	Aktywacja	Włącz wtyczkę	Brak błędu; w menu pojawia się „Nasze motory"
2	Podstrona	Wejść na /nasze-motory/	Nagłówek, wstęp, filtry, siatka kart z danymi z bazy
3	Karta	Obejrzyj kartę	Miniatura (hotlink), nazwa, cena, rok/przebieg/pojemność/paliwo
4	Filtr marka/paliwo/rok	Wybierz i „Filtruj"	Lista zawężona; URL zawiera ?polea_marka=...
5	Filtr ceny	„Cena do" = np. 30000	Tylko oferty <= 30000
6	Paginacja	Gdy >12 ofert – kliknij stronę 2	?polea_str=2, okno stron + wielokropek, filtry zachowane
7	Widok pojedynczy	Kliknij kartę	Ładny URL /nasze-motory/<lot_id>; galeria, opis, dane techniczne, „Informacje o aukcji", FAQ, „Podobne motocykle"
8	H1 = nazwa pojazdu	Widok pojedynczy -> źródło	<h1> = „Marka Model Rok" (podmieniony filtrem the_title)
9	CTA źródła	Przycisk „Zobacz aukcję na poleasingowe.pl"	target=_blank rel="noopener nofollow", prowadzi do źródła
10	Powrót	„<- Wróć do listy"	Wraca na /nasze-motory/
11	Cache	Odśwież listę 2x	Drugie wejście szybsze (transient 5 min)
12	Panel admina	Ustawienia -> Motocykle (lub menu wtyczki)	„Połączono. Motocykli w bazie: N"; przycisk czyszczenia cache (nonce)

4.3. Weryfikacja SEO (widok pojedynczy – „Pokaż źródło strony")

Element	Czego szukać
<title>	„Marka Model Rok – cena \
<meta name="description">	zsyntetyzowany opis (rok, przebieg, pojemność, moc, paliwo)
<link rel="canonical">	ładny URL tego motocykla (nie lista)
Open Graph	og:type=product, og:title/description/url/image
Twitter	twitter:card = summary_large_image gdy jest zdjęcie
product:price:amount + :currency=PLN	gdy jest cena
JSON-LD (application/ld+json w stopce)	@graph: Product/Motorcycle + Offer + BreadcrumbList + FAQPage
noindex	dla aukcji zakończonej lub nieistniejącej -> robots: noindex, follow

Lista: og:type=website, opis kolekcji, JSON-LD CollectionPage + ItemList + WebSite + Organization.

Widok filtrowany/paginowany -> robots: noindex, follow + canonical do bazowej listy.

Narzędzia zewnętrzne:

- Rich Results Test (search.google.com/test/rich-results) – wklej HTML widoku pojedynczego -> Product i FAQ wykryte, bez błędów.
- Sitemap: /wp-sitemap.xml -> sekcja polea_motocykle z URL-ami aktywnych ofert.
- Lighthouse (DevTools) – Performance/SEO/Accessibility na liście i szczegółach (LCP: pierwsza karta ma fetchpriority=high).

4.4. Przypadki brzegowe (klient NIGDY nie widzi błędu)

Scenariusz	Jak wywołać	Oczekiwane
Brak konfiguracji bazy	Zakomentuj POLEA_DB_* w wp-config.php (lub zatrzymaj db)	Strona pokazuje „Oferta motocykli będzie dostępna wkrótce." – żadnego błędu PHP
Pusta baza	Wyczyść polea_motocykle	„Brak motocykli spełniających kryteria."